BailAI

AI Criminal Litigation Assistant — Project README

1. Project Overview

BailAI is a professional legal technology platform focused on helping lawyers manage criminal litigation workflows and, in later phases, use AI-assisted tools for bail and case analysis. The project is structured as a full-stack application with a Python/FastAPI backend and a Next.js/React frontend.

2. Current Technology Stack

Layer	Technology
Frontend	Next.js 16.3.2, React 19.2.8, TypeScript, Tailwind CSS
Frontend API	Axios
Frontend data	TanStack React Query
Forms	React Hook Form + Zod + @hookform/resolvers
UI utilities	Tailwind Merge, CLSX, Lucide React
Backend	Python + FastAPI
API server	Uvicorn
ORM	SQLAlchemy
Database	PostgreSQL
Migrations	Alembic
Authentication	JWT + password hashing
Version control	Git + GitHub
Environment	Ubuntu Linux

3. Repository Structure

apps/api — FastAPI backend, authentication, case management, SQLAlchemy models and Alembic migrations.
apps/web — Next.js frontend, dashboard, authentication pages, case-management UI and API integration.
packages — Shared workspace packages reserved for reusable project functionality.
docs — Project documentation.
data — Project data resources.
models — Model-related resources.
scripts — Development and automation scripts.

4. Backend

The backend currently provides:

• Health endpoint
• Lawyer registration
• Lawyer login

• JWT access-token generation
• Current-user authentication dependency
• Protected case endpoints
• PostgreSQL persistence through SQLAlchemy
• Alembic database migrations

Current API route groups

/auth/register — register a lawyer
/auth/login — authenticate and receive JWT token
/cases/ — protected case-management endpoints
/ — API status/root endpoint

5. Frontend

The frontend uses Next.js App Router and is being developed as a professional legal dashboard. The current foundation includes a dashboard shell, sidebar, navbar, login/register pages, case pages, Axios API client and TanStack React Query provider.

Frontend architecture

app/ — Next.js routes and pages
components/layout/ — dashboard shell, navbar and sidebar
lib/api/ — centralized API client
services/ — frontend API service modules such as auth and cases
providers/ — application providers such as React Query

6. Authentication Flow

1. A lawyer registers through the frontend.
2. FastAPI validates the request and stores the password hash in PostgreSQL.
3. The lawyer logs in using email and password.
4. The backend verifies the password and returns a JWT access token.
5. The frontend stores the access token in localStorage.
6. Axios automatically sends the token as a Bearer Authorization header for API requests.
7. Protected case routes use the current authenticated user to restrict access.

7. Case Management

The current case-management foundation supports creating, listing, retrieving, updating and deleting cases. Cases are associated with the authenticated lawyer so that a lawyer's case list can be isolated from other users.

8. Local Development

Backend

From **apps/api**, activate the Python virtual environment and run:

```
source .venv/bin/activate
uvicorn app.main:app --reload
```

The API is currently configured to run at **http://127.0.0.1:8000**. FastAPI documentation is available at **/docs**.

Frontend

From **apps/web**, run:

```
pnpm dev
```

The Next.js application is currently configured to run at **http://localhost:3000**.

9. Environment Configuration

The frontend uses **.env.local** for the backend API URL:

```
NEXT_PUBLIC_API_URL=http://127.0.0.1:8000
```

Secrets such as JWT configuration and database credentials should remain in environment variables and must not be committed to GitHub.

10. Database & Migrations

PostgreSQL is the primary relational database. SQLAlchemy is used for ORM/database interaction and Alembic is used to version schema changes. Database changes should be introduced through migrations rather than manual production schema edits.

11. Git & GitHub Workflow

The project is maintained in Git and pushed to GitHub. The development workflow is intentionally terminal-driven: changes are created, updated, removed or replaced using commands, followed by validation, Git review, commit and push.

Recommended workflow

1. Inspect current project state.
2. Make the smallest correct change.
3. Run relevant checks/tests.
4. Review git diff/status.
5. Commit with a clear message.
6. Push to GitHub.
7. Confirm the working tree is clean.

12. Quality & Engineering Principles

BailAI should be developed as a professional production-oriented SaaS rather than as a collection of disconnected demos. Backend completion and end-to-end feature completion should be treated as separate milestones. Every feature should be validated across database → API → authentication/authorization → frontend service → UI → user workflow.

Priority principles: simple architecture, clear separation of concerns, strong authentication, lawyer-specific data isolation, typed frontend APIs, validated forms, migration-based database changes, clean UI/UX, maintainable code and disciplined Git history.

13. Current Project Status

The repository currently has a working FastAPI backend, PostgreSQL integration, JWT authentication, lawyer registration/login, protected case-management routes, and a Next.js frontend foundation with dashboard and authentication/case UI. The next development stages should focus on completing the case workflow, improving the user experience, strengthening authorization/error handling, and then introducing the core AI legal-assistance capabilities.

14. Important Security Note

BailAI handles potentially sensitive legal information. Production development must therefore include strong secret management, HTTPS, secure token strategy, authorization checks, input validation, auditability, appropriate database protection, logging without exposing sensitive information, and careful handling of client identifiers and case documents.

15. Product Direction

The long-term BailAI platform can evolve into an AI-assisted criminal litigation workspace containing case management, document management, legal research assistance, bail analysis, drafting assistance, case timelines, evidence/document intelligence, lawyer productivity tools and other legally focused AI workflows. AI features should assist lawyers and present grounded, reviewable outputs rather than replace professional legal judgment.

Document purpose: This README consolidates the project information into one place so the project architecture, technologies, workflow and current status remain clear for development and mentor/audit review.